Capítulo 8: Personalización de Git

Tutorial de Git basado en Pro Git (2.ª ed.) · Chacon y Straub · Elaborado por Ricardo · UNIAGRARIA

Introducción

Git puede adaptarse al estilo de trabajo de cada persona o equipo mediante opciones de configuración, atributos por tipo de archivo y scripts que se ejecutan automáticamente (hooks).

8.1 Configuración de Git

```
git config --global core.editor "nano"
git config --global commit.template ~/.gitmessage.txt
git config --global core.pager 'less -F -X'
git config --global user.signingkey <ID>
git config --global core.excludesfile ~/.gitignore_global
git config --global core.autocrlf true      # Windows: CRLF al sacar, LF al
confirmar
git config --global core.autocrlf input     # Linux/macOS
git config --global core.whitespace trailing-space,space-before-tab
git config --global color.ui auto
git config --global push.default simple
git config --global pull.rebase false
```

Una plantilla de commit (commit.template) ayuda a que todo el equipo siga el mismo formato en los mensajes. Los valores se guardan en ~/.gitconfig (global) o en .git/config (local).

8.2 Git Attributes

El archivo .gitattributes define comportamientos por ruta o patrón.

```
# .gitattributes
*.docx diff=word        # usar un filtro de comparación para Word
*.png binary            # tratar como binario
*.sh text eol=lf        # forzar finales de línea LF
database.xml merge=ours
*.zip export-ignore     # excluir de git archive
```

8.3 Puntos de enganche en Git (hooks)

Los hooks son scripts ejecutables en .git/hooks que Git lanza ante ciertos eventos. Los ejemplos vienen con extensión .sample; para activarlos basta quitarla y darles permiso de ejecución.

- Del lado del cliente: pre-commit, prepare-commit-msg, commit-msg, post-commit, pre-rebase, post-checkout, post-merge, pre-push.
- Del lado del servidor: pre-receive, update, post-receive.

```
#!/bin/sh
# .git/hooks/commit-msg: rechaza mensajes de menos de 10 caracteres
if [ $(wc -c < "$1") -lt 10 ]; then
  echo "El mensaje de commit es demasiado corto" >&2
  exit 1
fi
```

> **Nota:** Si un hook de cliente termina con código distinto de cero, la operación se cancela. Los hooks no se versionan con el repositorio; para compartirlos se guardan en una carpeta del proyecto y se configura core.hooksPath.

8.4 Ejemplo de implantación de una política

El libro muestra cómo un hook update en el servidor puede imponer una política: verificar que los mensajes sigan un formato, que cada usuario solo modifique las rutas permitidas (lista de control de acceso) y que solo se hagan push fast-forward. Los hooks del lado del cliente complementan esa política, pero no son obligatorios para el desarrollador, mientras que los del servidor sí lo son.

Resumen

La configuración, los atributos y los hooks permiten ajustar Git a las necesidades del equipo: editor, plantillas de mensajes, finales de línea, tratamiento de binarios y validaciones automáticas antes o después de eventos como commit o push.

Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2 — Capítulo 8.